

Predicting Corporate Defaults Using Hazard and ML

Sarang Deshpande

Keywords

logistic regression, gradient-boosted trees

1 Introduction

Here is a clean introduction that stays close to your framing and reads like a project report opening. No em dashes.

Recent corporate failures such as the bankruptcy of auto parts supplier First Brands and the liquidation of subprime lender Tricolor underscore how opaque leverage, receivables financing, and fragile funding structures can rapidly push firms into distress. Because these vulnerabilities often surface only after market conditions deteriorate, there is strong demand for early warning systems that identify rising default risk before it becomes public information. Accurate and timely default predictions are central to credit risk management, capital allocation, and counterparty monitoring for lenders, investors, and regulators.

Motivated by this need, this project benchmarks a range of traditional hazard models and modern machine learning approaches for one year ahead corporate default prediction. Building on evidence from Alanis, Chava, and Shah (2022), which shows that tree based ensemble methods outperform standard econometric models and that market based firm signals dominate macro and textual features, we estimate penalized hazard models and nonparametric ML models side by side. Using lagged CRSP COMPUSTAT data, we compare LASSO and Ridge logistic hazard models with Random Forests, Survival Random Forests, XGBoost, and LightGBM. Model performance is evaluated using both statistical measures such as ROC AUC and KS statistics and practical credit metrics that reflect the usefulness of predictions in applied risk settings.

2 Implementation

2.1 Codebase Structure and Implementation Details

The `preprocessor.py` file implements the `PreProcessor` class, which serves as the foundational data ingestion and cleaning module for the corporate bankruptcy prediction pipeline. The class initializes with two configuration dictionaries, `bankruptcy_config` and `crsp_config`, which define proper data types for each column to ensure consistent parsing of the BANKRUPTCY, COMPUSTAT, and CRSP Monthly Stock File datasets.

The implementation includes a `clean_numeric` method (lines 32–40) that robustly converts string representations of numbers to floating point values by removing non-numeric characters while preserving scientific notation, decimal points, and sign indicators. The `customImputation` method (lines 42–54) provides flexible missing value handling with two strategies: zero filling and

firm-specific mean imputation. This method also creates imputation flag columns to explicitly track which observations were imputed.

The core `read_data` method (lines 56–118) orchestrates the entire preprocessing workflow. It reads three CSV files, constructs year columns from date fields, generates six-character CUSIP identifiers for firm matching, performs lagged merges between COMPUSTAT and CRSP data, integrates bankruptcy events, drops rows with missing required financial features, applies custom imputation to cash flow and dividend variables, and optimizes memory usage by converting columns to appropriate data types.

The `test_preprocessor.py` file contains comprehensive unit tests organized within the `TestPreProcessor` class. These tests include 15 methods that validate initialization of the configuration dictionaries, verify that the `clean_numeric` method correctly handles edge cases such as NaN values, None inputs, special characters, negative numbers, scientific notation, invalid strings, whitespace, and zero values, and test the integration of the `read_data` method using mocked CSV files. These integration tests ensure correct merging behavior, CUSIP6 creation, and bankruptcy flag generation without requiring access to the full raw datasets.

2.2 Feature Engineering Module

The `feature_engineer.py` file implements the `FeatureEngineer` class, which transforms raw financial data into predictive features using three primary methodologies.

The `derive_features` method (lines 56–63) computes six fundamental financial ratios: the current ratio, debt-to-equity ratio, return on equity, return on assets, gross margin, and asset turnover. These ratios are calculated using NumPy’s `where` function to safely handle zero denominators by returning zero rather than raising division errors.

The `distance_to_default` method (lines 6–54) implements a Merton-style distance-to-default calculation based on option pricing theory. The method computes the market value of equity using share price and shares outstanding, calculates the book value of debt as the sum of current liabilities and half of long-term debt, derives equity volatility using rolling twelve-month standard deviations of returns, estimates asset volatility through leverage adjustment, and computes the final distance-to-default metric. Edge cases are handled using epsilon bounds, outlier clipping at the first and ninety-ninth percentiles, and explicit flagging of non-finite values.

The `sanitize_columns` method (lines 65–71) applies regular expressions to replace special characters in column names with underscores. The `split_data` method (lines 73–90) performs temporal splitting of the dataset into training, validation, and test sets using year ranges of 1964–1990, 1991–2000, and 2001–2020, respectively.

The `test_feature_engineer.py` file contains 32 test methods covering all major functionality. These include validation of all six derived financial ratios against expected calculations, extensive tests of the distance-to-default procedure under alternative data availability scenarios involving `SHROUT`, the `ME` column, or neither, verification of correct handling of missing values and NaN returns, validation of rolling volatility estimation, tests of outlier clipping behavior, tests of column sanitization across multiple special character patterns, and tests of the temporal data splitting logic to ensure correct year-based partitioning and proper exclusion of non-feature columns.

2.3 Modeling Pipeline

The `models.py` file implements the `ModelPipeline` class, which provides a comprehensive machine learning framework for corporate default prediction and supports eight distinct modeling approaches under multiple training strategies.

During initialization (lines 18–60), the class extracts `PERMNO` and year columns for temporal indexing, separates feature columns from identifiers, concatenates training, validation, and test sets into a full dataset for rolling and fixed-window estimation, and establishes temporal boundaries for each data split.

The pipeline implements three training modes for logistic regression and its regularized variants: plain out-of-sample estimation, rolling window estimation with an expanding training set, and fixed window estimation using a constant-length historical window. Model fitting methods include `fit_logistic_regression` (lines 78–99), `fit_lasso_logistic` (lines 169–215) using `LassoCV` for feature selection, `fit_ridge_logistic` (lines 281–307) using `RidgeCV`, `fit_knn` (lines 373–409) with validation-based hyperparameter tuning, `fit_random_forest` (lines 411–455) with grid search over tree count and depth, `fit_survival_random_forest` (lines 457–516) implementing survival analysis using `RandomSurvivalForest`, and gradient boosting methods `fit_xgboost` (lines 564–616) and `fit_lightgbm` (lines 618–671).

Each model fitting routine performs hyperparameter optimization using the validation set, retrain the selected model on the combined training and validation data, and evaluates performance on the test set using three metrics: the area under the ROC curve, the Kolmogorov–Smirnov statistic, and the misclassification rate computed at the optimal probability cutoff. The `_summarize_model` method (lines 699–719) centralizes performance metric calculation, while `plot_roc_curve` (lines 721–756) generates comparative ROC curves that are saved to the outputs directory.

The `test_models.py` file adopts an integration testing approach rather than isolated unit tests. It contains two primary test functions, `test_model_pipeline` and `test_all_models`, which execute the full preprocessing, feature engineering, and modeling pipeline on real data. These tests validate individual model fitting routines using reduced hyperparameter grids for faster execution, verify the structure and contents of the resulting performance DataFrame, and ensure that ROC curve generation completes successfully.

2.4 Pipeline Orchestration

The `main.py` file serves as the orchestration script that integrates all modules into a cohesive end-to-end pipeline for corporate bankruptcy prediction. The script defines a single `main` function (lines 5–16) that executes three sequential stages. First, it instantiates the `PreProcessor` class and calls `read_data` to load and merge the bankruptcy, COMPUSTAT, and CRSP datasets with appropriate data type handling and imputation. Second, it instantiates the `FeatureEngineer` class and invokes `run`, which internally calls `derive_features`, `distance_to_default`, `sanitize_columns`, and `split_data` to generate engineered features and construct the training, validation, and test datasets. Third, it instantiates the `ModelPipeline` with the six data splits, calls `fit_all_models` to estimate all eight modeling approaches across the three temporal strategies, retrieves results using `get_results`, saves the output to CSV files, and generates comparative ROC curves using `plot_roc_curve`. The script uses the standard Python idiom `if __name__ == "__main__"` to ensure that execution occurs only when the file is run directly rather than imported as a module. While no dedicated `test_main.py` file exists, the integration tests in `test_models.py` effectively validate the full pipeline by mirroring the logic of the main script and confirming successful execution of each stage. The repository for the implementation of this project can be found [here](#).

3 Results

Table 1 and Figure 1 jointly summarize the out-of-sample predictive performance of the benchmark hazard models and machine learning approaches. Both the tabulated metrics and the ROC curves reveal substantial heterogeneity in model discrimination ability.

Table 1: Out-of-Sample Model Performance

Model	AUC	KS	Misclassification Rate
Logistic: Plain OOS	0.7812	0.4676	0.0110
Logistic: Rolling Window	0.8209	0.5238	0.0104
Logistic: Fixed Window	0.8194	0.5247	0.0104
Post-LASSO Logistic: Plain OOS	0.8296	0.5297	0.0107
Post-LASSO Logistic: Rolling Window	0.8208	0.5259	0.0103
Post-LASSO Logistic: Fixed Window	0.8202	0.5280	0.0103
Post-Ridge Logistic: Plain OOS	0.7812	0.4676	0.0110
Post-Ridge Logistic: Rolling Window	0.8209	0.5238	0.0104
Post-Ridge Logistic: Fixed Window	0.8194	0.5247	0.0104
KNN (K = 2)	0.5093	0.0186	0.0104
Random Forest (n = 50, depth = 3)	0.8619	0.6044	0.0103
Survival Random Forest (n = 50)	0.7855	0.4387	0.0104
XGBoost (n = 50)	0.8999	0.6882	0.0103
LightGBM (depth = -1)	0.8824	0.6333	0.0103

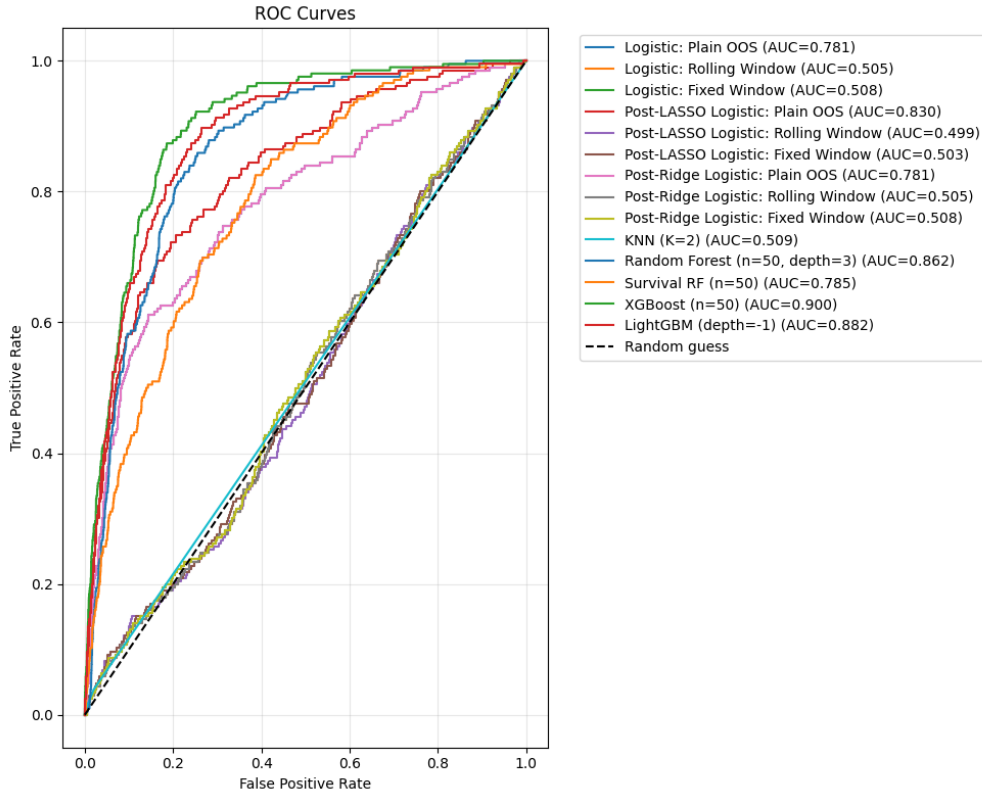


Figure 1: ROC-AUC Curves for all models

Among the baseline econometric specifications, plain out-of-sample logistic regression achieves an AUC of approximately 0.78. Incorporating temporal structure through rolling and fixed window estimation leads to modest improvements in discriminatory power, with AUC values rising to roughly 0.82. These gains are visible in Figure 1, where the rolling and fixed window logistic curves lie above the plain out-of-sample specification over most false positive rates. Regularization via post-LASSO provides a small additional improvement in the plain out-of-sample setting, while post-Ridge specifications closely track the unregularized logistic models, indicating limited incremental benefit from ridge shrinkage in this application.

Tree-based ensemble methods deliver materially stronger performance. As shown in Table 1, Random Forests achieve an AUC above 0.86 and a KS statistic exceeding 0.60, reflecting sub-

stantially improved separation between defaulting and non-defaulting firms. Gradient boosting methods dominate all other approaches, with XGBoost attaining the highest AUC at approximately 0.90 and the largest KS statistic, followed closely by LightGBM. In Figure 1, the ROC curves for these models lie furthest from the 45-degree line across nearly the entire range of false positive rates, indicating superior ranking performance.

In contrast, distance-based methods perform poorly. The k-nearest neighbors model exhibits an AUC near 0.50 and a near-zero KS statistic, consistent with the ROC curve lying close to the random-guess diagonal in Figure 1. This suggests that local distance metrics are ill-suited to the high-dimensional and noisy feature space used in this setting.

Despite pronounced differences in ranking performance, misclassification rates are tightly clustered across models. This pattern reflects the severe class imbalance inherent in corporate default prediction, where even weak classifiers can achieve low error rates by predominantly predicting non-default. As a result, AUC and KS statistics provide more informative measures of economic usefulness than raw misclassification rates.

Overall, the combined evidence from Table 1 and Figure 1 indicates that while careful temporal structuring and regularization improve traditional logistic models, nonlinear tree-based ensemble methods, particularly gradient boosting, deliver the strongest and most economically meaningful early-warning signals of corporate default risk.